

AFRILANG Stdlib

std/c/complz77

Français. Bibliothèque AFRILANG 'std/c/complz77' (niveau complex, catégorie complex). Elle expose 6 fonction(s) : lz77Encode, lz77Decode, lz77Ratio, lz77Size, lz77Compressed, lz77RoundTrip.

```
'import "std/c/complz77"' · 'use complz77'
```

```
- 'lz77Encode(data list number) → list number' - 'lz77Decode(encoded  
list number) → list number' - 'lz77Ratio(data list number) → number' -  
'lz77Size(data list number) → number' - 'lz77Compressed(data list num-  
ber) → number' - 'lz77RoundTrip(data list number) → bool'
```

English.

AFRILANG library 'std/c/complz77' (tier complex, category complex). Exports 6 function(s): lz77Encode, lz77Decode, lz77Ratio, lz77Size, lz77Compressed, lz77RoundTrip.

```
'import "std/c/complz77"' · 'use complz77'
```

```
- 'lz77Encode(data list number) → list number' - 'lz77Decode(encoded  
list number) → list number' - 'lz77Ratio(data list number) → number' -  
'lz77Size(data list number) → number' - 'lz77Compressed(data list num-  
ber) → number' - 'lz77RoundTrip(data list number) → bool'
```

Catégorie / Category	Modules complex (c/)
Niveau / Tier	complex
Fonctions / Functions	6

June 16, 2026

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/complz77' (niveau complex, catégorie complex). Elle expose 6 fonction(s) : lz77Encode, lz77Decode, lz77Ratio, lz77Size, lz77Compressed, lz77RoundTrip.

```
'import "std/c/complz77"' · 'use complz77'
```

```
- `lz77Encode(data list number) → list number`
- `lz77Decode(encoded list number) → list number`
- `lz77Ratio(data list number) → number`
- `lz77Size(data list number) → number`
- `lz77Compressed(data list number) → number`
- `lz77RoundTrip(data list number) → bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/complz77"
use complz77
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- lz77Encode(data list number) → list•
lz77Decode(encoded list number) → list•
lz77Ratio(data list number) → number•
lz77Size(data list number) → number•
lz77Compressed(data list number) → number•
lz77RoundTrip(data list number) → bool

4. Exemples d'utilisation

```
import "std/c/complz77"
use complz77
```

```
create result = lz77Encode(/* args */)
say result
```

```
create result = lz77Decode(/* args */)
say result
```

```
create result = lz77Ratio(/* args */)
say result
```

```
create result = lz77Size(/* args */)
say result
```

```
create result = lz77Compressed(/* args */)
say result
```

```
create result = lz77RoundTrip(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/complz77"`` en tête de fichier.
- Lancez ``afrilang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module complz77

```
function lz77Encode(data list number) returns list number  
end
```

```
function lz77Decode(encoded list number) returns list number  
end
```

```
function lz77Ratio(data list number) returns number  
end
```

```
function lz77Size(data list number) returns number  
end
```

```
function lz77Compressed(data list number) returns number  
end
```

```
function lz77RoundTrip(data list number) returns bool  
end  
end
```

English

1. Overview

AFRILANG library 'std/c/complz77' (tier complex, category complex). Exports 6 function(s): lz77Encode, lz77Decode, lz77Ratio, lz77Size, lz77Compressed, lz77RoundTrip.

'import "std/c/complz77"' · 'use complz77'

```
- `lz77Encode(data list number) → list number`
- `lz77Decode(encoded list number) → list number`
- `lz77Ratio(data list number) → number`
- `lz77Size(data list number) → number`
- `lz77Compressed(data list number) → number`
- `lz77RoundTrip(data list number) → bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/complz77"
use complz77
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- lz77Encode(data list number) → list•
lz77Decode(encoded list number) → list•
lz77Ratio(data list number) → number•
lz77Size(data list number) → number•
lz77Compressed(data list number) → number•
lz77RoundTrip(data list number) → bool

4. Usage examples

```
import "std/c/complz77"
use complz77
```

```
create result = lz77Encode(/* args */)
say result
```

```
create result = lz77Decode(/* args */)
say result
```

```
create result = lz77Ratio(/* args */)
say result
```

```
create result = lz77Size(/* args */)
say result
```

```
create result = lz77Compressed(/* args */)
say result
```

```
create result = lz77RoundTrip(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/complz77"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module complz77

```
function lz77Encode(data list number) returns list number
end
```

```
function lz77Decode(encoded list number) returns list number
end
```

```
function lz77Ratio(data list number) returns number
end
```

```
function lz77Size(data list number) returns number
end
```

```
function lz77Compressed(data list number) returns number
end
```

```
function lz77RoundTrip(data list number) returns bool
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/complz77"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/complz77/>

Source (extrait)

```
module complz77

function lz77Encode(data list number) returns list number
end

function lz77Decode(encoded list number) returns list number
end

function lz77Ratio(data list number) returns number
end

function lz77Size(data list number) returns number
end

function lz77Compressed(data list number) returns number
```

```
end
```

```
function lz77RoundTrip(data list number) returns bool
```

```
end
```

```
end
```